Hash

Hash表

哈希表（Hash Table），也称为**散列表**，是一种极其高效的数据结构，用于存储键值对（Key-Value pairs）。它的核心优势在于能够在**平均 O(1)的时间复杂度**内完成数据的插入、删除和查找操作。**空间换时间**的典型代表，牺牲了一定的内存空间来换取极快的读写速度。

1. 核心原理

哈希表的工作原理基于**哈希函数（Hash Function）**：

1. **输入**：给定一个键（Key），例如字符串 "apple"。
2. **计算**：通过哈希函数将该键转换成一个整数索引（Index），例如 `hash("apple") = 3`。
3. **存储**：将这个键值对直接存储在底层数组（Array）的第 3 号位置。
4. **查找**：当需要查找 "apple" 时，再次计算 `hash("apple")` 得到 3，直接访问数组第 3 号位置即可，无需遍历整个数据集合。

2. 关键特性

A. 哈希冲突（Hash Collision）

由于哈希函数将无限可能的键映射到有限大小的数组索引上，不同的键可能会计算出相同的索引（例如 "apple" 和 "orange" 都算出索引 3）。这就是**哈希冲突**。
解决冲突的常见方法有：

- **链地址法（Chaining / Separate Chaining）**：在数组的每个位置上维护一个链表（或红黑树）。如果发生冲突，就将新元素添加到该位置的链表中。Java 的 `HashMap` 在链表过长时会转为红黑树以提高效率。
- **开放寻址法（Open Addressing）**：如果目标位置被占用，就按照某种探测规则（如线性探测、二次探测）寻找下一个空闲位置存入。

B. 负载因子（Load Factor）

负载因子 = 元素总数 / 数组容量。

- 当负载因子过高时，冲突概率增加，性能下降。
- 当负载因子超过阈值（通常是 0.75）时，哈希表会自动进行**扩容（Rehashing）**：创建一个更大的数组，并将所有旧元素重新计算哈希值并迁移到新数组中

C++技巧

- #include<unordered_map>
- `std::unordered_map<Key, Value>`: 键值对映射。用于统计频率、存储索引、缓存结果等。
- `std::unordered_set<Key>`: 仅存储键。用于快速去重、判断元素是否存在。

A. 访问元素的三种方式

1. `mp[key]` **(下标操作符)**:
   - **特点**: 如果 key 不存在，会**自动插入**一个默认值（如 int 为 0），并返回引用。
   - **场景**: 统计词频 (`mp[word]++`) 时非常方便。
   - **陷阱**: 在只读查找时使用它会意外增加元素，改变 map 大小，可能导致迭代器失效或逻辑错误。
2. `mp.find(key)` **(推荐用于查找)**:
   - **特点**: 返回迭代器。如果没找到，返回 `mp.end()`。**不会修改容器。**
   - **场景**: 判断是否存在，或获取值但不想插入新元素。
3. `mp.count(key)`:
   - **特点**: 返回 1 (存在) 或 0 (不存在)。对于 `unordered_multimap` 可能返回大于 1 的值。
   - **场景**: 仅需判断存在性，代码最简洁。
4. `mp.at(key)`:
   - **特点**: 如果 key 不存在，抛出 `std::out_of_range` 异常。
   - **场景**: 确定 key 一定存在，且希望出错时由异常机制处理。

C. 删除元素

- **按 Key 删除**: `mp.erase(key)` 返回删除的数量 (0 或 1)。

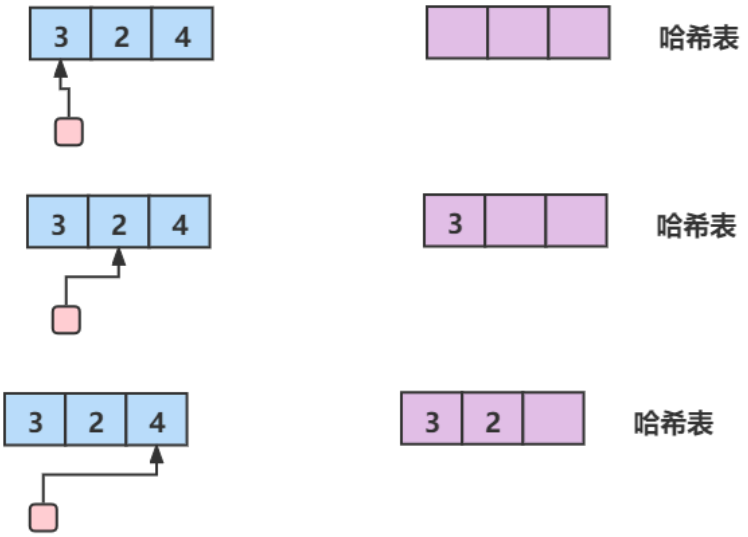
leetcode

两数之和

- 给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** *`target`* 的那 **两个** 整数，并返回它们的数组下标。
- 思路1：双循环暴力搜索，时间复杂度O(n^2)

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        for(int i=0;i < nums.size();i++){
            for(int j=i+1;j < nums.size();j++){ // 注意j下标
                if (nums[i]+nums[j]==target){
                    vector<int> v{i,j};
                    return v;
                }
            }
        }
        return vector<int>();
    }
};
```

- 思路2：**一边遍历，一边查找。**利用遍历可以构建哈希表，然后元素只需要**查找缺值**就能找到target了。




```
class Solution {
public:
    unordered_map<int,int> numToIndexMap;
    vector<int> twoSum(vector<int>& nums, int target) {
        for(int i=0; i< nums.size();i++){
            int findX = target - nums[i];// 补数，当前nums[i]需要多大的数字才能到target
            // 在hash表中查找findX是否存在
            if(numToIndexMap.count(findX)!=0){
                vector<int> v{i,numToIndexMap[findX]};
                return v;
            }
            numToIndexMap[nums[i]] = i; // 一边遍历，一边构建哈希表
        }
        return vector<int> ();
    }
};
```

-

字母异位词分组

- 一个字符串数组，将 **字母异位词（单词相同但是字母位置不同）** 组合在一起。

输入： strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

输出： [["bat"],["nat","tan"],["ate","eat","tea"]]

解释：

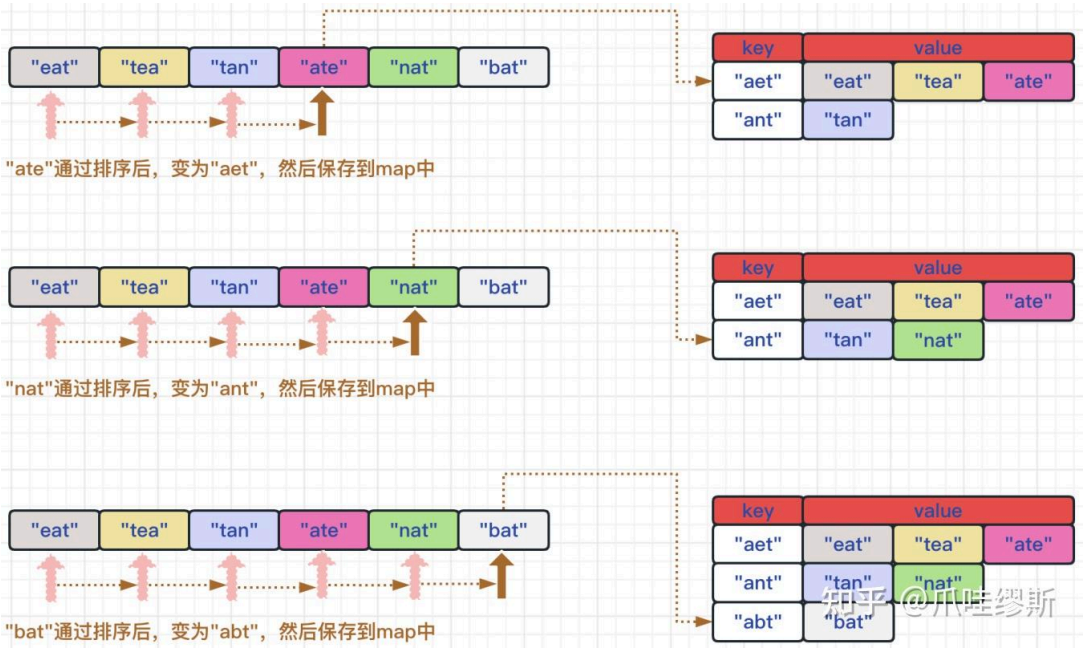
- 在 strs 中没有字符串可以通过重新排列来形成 "bat"。
- 字符串 "nat" 和 "tan" 是字母异位词，因为它们可以重新排列以形成彼此。
- 字符串 "ate" ，"eat" 和 "tea" 是字母异位词，因为它们可以重新排列以形成彼此。

- 思路：有点类似于one-hot编码，把字符串映射成26位的10001000这种特征字符串，然后构建一个特征字符串和字母异位词(string,vector)的hash表。

假设输入： `["eat", "tea", "tan"]`

1. 处理 "eat":
   - 统计：a:1, e:1, t:1, 其他:0。
   - 生成 Key `v`： `"10001000000000000001000000"` (简化表示，实际是字符累加)。
   - `mm["1...1...1..."]` = `["eat"]`
2. 处理 "tea":
   - 统计：a:1, e:1, t:1, 其他:0。
   - 生成 Key `v`：与上面完全相同。
   - `mm["1...1...1..."]` = `["eat", "tea"]` (追加)
3. 处理 "tan":
   - 统计：a:1, n:1, t:1, 其他:0。
   - 生成 Key `v`：不同（'n' 的位置有值，'e' 的位置没值）。
   - `mm["1...0...1...n_pos..."]` = `["tan"]`

最终结果： `[["eat", "tea"], ["tan"]]`




```
class Solution {
public:
    // 哈希表：Key为特征字符串(sigString)，Value为具有该特征的原始字符串列表
    unordered_map<string,vector<string>> sigStringToStrings;
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        // 构建hash table
        for(auto e: strs){
            // string sigString(26,'0');
            // for(auto ee :e) // 遍历字符串e，统计各位置字符的频率，构建其特征字符串
1001XXX
            // {
            //     int idx  = ee - 'a';
            //     sigString[idx]++;
            // }
```

```
            string sigString= e;
            sort(sigString.begin(),sigString.end()); // 特征字符串aet
            sigStringToStrings[sigString].push_back(e);
        }
        // 遍历hash table，返回结构
        vector<vector<string>> ans;
        for(auto e : sigStringToStrings){
            ans.push_back(e.second); // 将key对应的value压入ans中
        }
        return ans;
    }
};
```

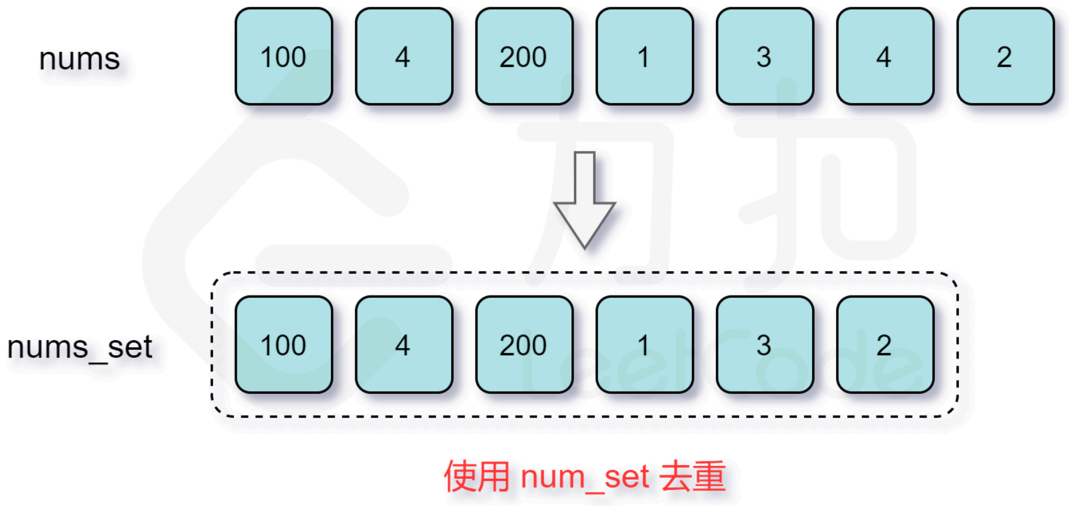
-

⭐ 最长连续序列

- 给定一个未排序的整数数组 `nums`，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

```
输入: nums = [100,4,200,1,3,2]
输出: 4
解释: 最长数字连续序列是 [1, 2, 3, 4]。它的长度为 4。
```

- 思路:




```
class Solution {
public:
    int res = 0; // 记录目前找到的最长连续序列长度
    unordered_set<int> uniNums; // 用于快速判断某个数字是否存在
    unordered_map<int,int> numsToValLength; // Key=数字，Value=以该数字开头的最长连续长度

    int update(int i){
        // 以该数字为起点的连续序列长度已经计算过，直接查Hash Table返回最长长度
        if ( numsToValLength.count(i) != 0 ) return numsToValLength[i];
        if ( uniNums.count(i) == 0 ) return 0; // 为起点数字不存在，则以它为起点的连续序列长度为0
```

```
        // 递归计算，以i为起点的最长连续序列=1+以1+1为起点的最长连续序列
        numsToValLength[i] = 1 + update(i+1);
        res = max(res,numsToValLength[i]); // 更新最长的连续序列
        return numsToValLength[i];

    }

    int longestConsecutive(vector<int>& nums) {
        // 数字去重，记录到hash table
        for (int i=0; i < nums.size(); i++){
            uniNums.insert(nums[i]);
        }
        // 遍历去重后的数字，得到以每个元素为起点的连续序列的最长长度
        for(int i=0; i < nums.size(); i++){
            update(nums[i]);
        }

        return res;
    }
};
```

-